

Развёртывание и настройка NFS-сервера для сетевого хранения данных

Курсовой проект по курсу «Администрирование Unix-подобных систем»

Григорян А. А. Карлашов А. В. Кроливец И. С.

ЮФУ · Физический факультет

Ростов-на-Дону, 2026

- 1 Актуальность и постановка задачи
- 2 Распределение ролей
- 3 Теоретические основы
- 4 Архитектура и сетевая инфраструктура
- 5 Автоматизация (IaC)
- 6 Безопасность

Ключевые проблемы

- **Изолированное хранение данных** — каждый контейнер хранит данные отдельно, что порождает дублирование
- **Масштабируемость** — требуется единая точка хранения для всех рабочих узлов
- **Configuration drift** — ручная настройка приводит к расхождению конфигураций

Решение

NFSv4

Единый порт 2049/TCP
Kerberos-ready

IaC подход

Docker Compose
bootstrap.sh
CI/CD пайплайн

Цель работы

Проектирование, развёртывание и защита NFS-сервера сетевого хранения данных в контейнеризированной среде под управлением ОС Linux

Задачи для достижения цели:

- 1 Развёртывание NFS-сервера в Docker с персистентным томом данных
- 2 Подключение нескольких клиентских контейнеров через Docker Volume (type: nfs4)
- 3 Реализация сетевой изоляции (storage-net) и hardening (UFW, cap_drop)
- 4 Написание скриптов тестирования (test_nfs.sh) и бэкапа
- 5 Настройка CI/CD пайплайна и мониторинга Prometheus + Grafana

Распределение ролей и индивидуальный вклад

Григорян Ашот

Инженер по автоматизации

- Структура репозитория, Git
- Docker Compose манифест
- `bootstrap.sh` — хост
- Dockerfile (non-root)
- Makefile: `up/down/test`

Карлашов Андрей

Системный администратор / SRE

- Настройка `/etc/exports`
- NFSv4: `fsid=0`, порт 2049
- Клиентские тома (type: `nfs4`)
- healthcheck: `showmount`
- HA-тест: `failover`

Кроливец Иван

Инженер по безопасности

- `hardening.sh`: UFW
- Изоляция `storage-net`
- Prometheus + Grafana
- Дашборды: I/O, метрики
- CI/CD: GitHub Actions

Ключевые свойства NFSv4:

- Единственный порт 2049/TCP — упрощает правила брандмауэра
- Сессионная модель со стейтом
- Kerberos / RPCSEC_GSS
- pNFS (v4.1) — параллельный доступ

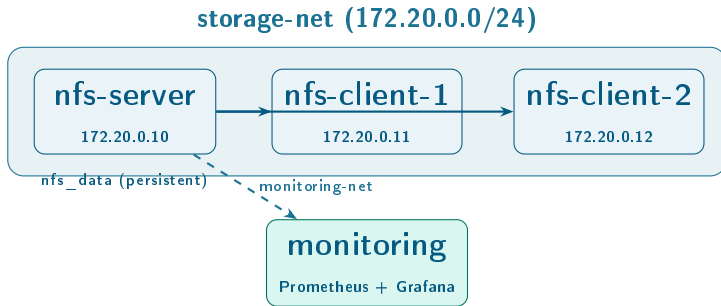
Опции /etc/exports:

rw/ro, sync/async, root_squash,
no_subtree_check, fsid=0

Сравнение альтернатив

Технология	Вывод
NFS	Оптимален
Samba	Избыточен
GlusterFS	Избыточен
iSCSI	Другой класс

Архитектура распределённого кластера



storage-net — NFS-трафик изолирован, порт 2049 закрыт снаружи

monitoring-net — Grafana на порту 3000 с HTTP-auth

Virtual Network storage-net

- nfs-server: fixed IP 172.20.0.10
- Клиенты: .11 и .12
- Порт 2049 закрыт из внешнего мира

Мониторинг периметра

- Grafana: порт 3000, HTTP-auth
- Prometheus: доступ только через localhost

Правила UFW

```
ufw default deny incoming
ufw allow 22/tcp
ufw allow from \
172.20.0.0/24 \
to any port 2049
```

Защита от утечек трафика

NFS локализован внутри storage-net

Автоматизация развёртывания: Docker Compose

```
services:
  nfs-server:
    image: erichough/nfs-server:latest
    privileged: true
    environment:
      NFS_EXPORT_0: '/exports/shared *(rw,sync,no_subtree_check,fsid=0)'
    volumes:
      - nfs_data:/exports/shared
    networks:
      storage-net:
        ipv4_address: 172.20.0.10
    healthcheck:
      test: ["CMD", "showmount", "-e", "localhost"]
      interval: 30s   timeout: 10s   retries: 5

  nfs-client-1:
    image: alpine:3.19
    depends_on:
      nfs-server:
        condition: service_healthy
    volumes:
      - nfs_shared:/mnt/shared
    networks: [storage-net]
```

Монтирование NFS через Docker Volume (NFSv4)

```
volumes:  
  nfs_shared:  
    driver: local  
    driver_opts:  
      type: nfs4  
      o: "addr=172.20.0.10,rw,sync,nfsvers=4,hard,intr"  
      device: ":/exports/shared"  
    nfs_data:  
      driver: local  
networks:  
  storage-net:  
    driver: bridge  
  ipam:  
    config:  
      - subnet: 172.20.0.0/24  
  monitoring-net:  
    driver: bridge
```

Первичная подготовка среды (bootstrap.sh)

```
#!/bin/bash
# @file bootstrap.sh  @brief Host initialization
set -euo pipefail

EXPORT_DIR="${EXPORT_DIR:-/opt/nfs/shared}"
BACKUP_DIR="${BACKUP_DIR:-/opt/nfs/backups}"

# Initialization of target mount directories
for dir in "${EXPORT_DIR}" "${BACKUP_DIR}"; do
if [[ ! -d "${dir}" ]]; then
mkdir -p "${dir}"
chown nobody:nogroup "${dir}"
chmod 777 "${dir}"
echo "LOG: Created ${dir}"
fi
done

# Environment config verification
if [[ ! -f ".env" ]]; then
cp .env.example .env
echo "WARN: .env generated!"
fi
```

Обеспечение безопасности (Hardening)

Уровень хоста (hardening.sh)

- UFW: default deny incoming
- 22/TCP — SSH, только доверенные IP
- 2049/TCP — строго для 172.20.0.0/24

Уровень контейнеров

- cap_drop: [ALL] у клиентов
- read_only: true для корня ФС
- Запуск процессов от USER appuser
- privileged: true для сервера — необходимое исключение ядерного уровня

Уровень NFS

root_squash: привилегированные запросы

4 стадии автоматической валидации:

- 1 **Сетевой скан** — проверка экспорта через `showmount (timeout 60s)`
- 2 **Монтирование** — парсинг вывода `df -h` в клиентах
- 3 **I/O валидация** — сквозная запись из `client-1` и чтение из `client-2`
- 4 **НА-сценарий** — рестарт сервера и проверка восстановления дескрипторов файлов

Резервное копирование

`rsync -archive -checksum`

Инкрементальный бэкап (7 суток):

```
find "${BACKUP_DIR}" -mtime +7 -  
exec rm {} +
```

Спецификация

pr-validation.yml

(Триггер: Pull Request в main):

- 1 **yamllint** — валидация манифестов
- 2 **ShellCheck** — статический анализ bash
- 3 **Hadolint** — оптимизация Dockerfile
- 4 **Инфратест** — запуск test_nfs.sh

Защита веток

Слияние с веткой main заблокировано до прохождения тестов

Анализ	3 линтера
Тест	Стенд в облаке

Сбор телеметрии:

- `node_exporter` — системные метрики хоста (CPU, RAM, Disk I/O)
- `cAdvisor` — производительность контейнеров
- **Prometheus** — TSDB, scrape: 15s

Grafana дашборды (laC):

- NFS метрики: ops/s, bytes r/w
- Ресурсы: утилизация лимитов

Алертинг

Оповещение при дауне > 30с

Результат	Статус
Развёртывание NFS Replica Set в Docker Compose	✓ Выполнено
Автоматизация: bootstrap.sh и Makefile	✓ Выполнено
Многоуровневый hardening (UFW, cap_drop, RO)	✓ Выполнено
Интеграционные тесты и HA-тестирование	✓ Выполнено
Мониторинг Prometheus + Grafana	✓ Выполнено
CI/CD пайплайн GitHub Actions	✓ Выполнено

Ключевые достижения

Концепция **IaC** полностью исключила дрейф конфигураций • Сетевая безопасность.
Наблюдаемость через Prometheus/Grafana • Контроль стабильности через CI/CD.

Спасибо за внимание!

Григорян А. А. Карлашов А. В. Кроливец И. С.

ЮФУ · Физический факультет · 2026

Тема проекта

Развёртывание и настройка NFS-сервера для сетевого хранения данных

Проверил: Тимошенко Павел Евгеньевич